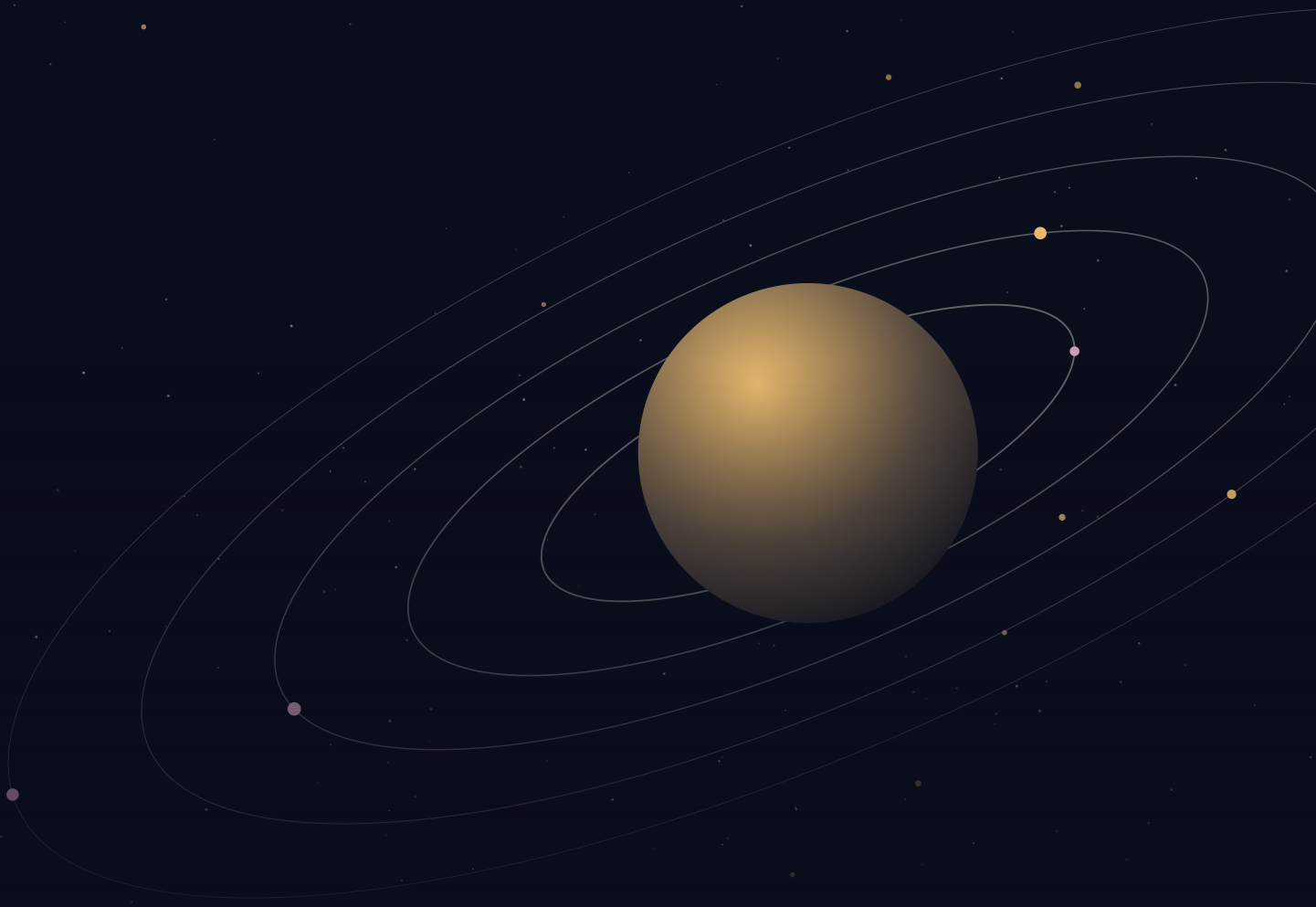


PHYB57 · 第 12 章 · 深色讲解版

Python 可视化与绘图

从 plot(x, y) 到三维曲面、地图与动画

2026-11-07

目录

01 开篇：先弄清一张图是怎么长出来的 3

02 二维绘图：`plot(x, y)` 的真相 5

03 让图能被读懂：标题、标签、图例与样式 8

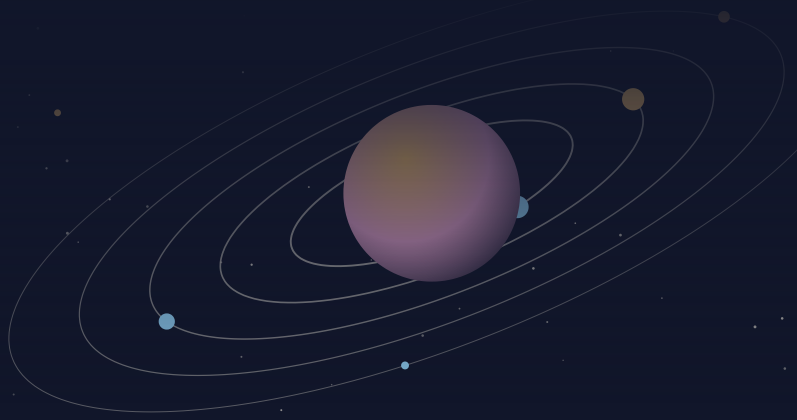
04 三维绘图：从平面走进空间 11

05 地图：把数据放回地球上 15

06 Cartopy 进阶：区域、要素与分辨率 18

07 动画：让时间成为第三个维度 21

08 速查、常见坑与自测 25



01

开篇：先弄清一张图是怎么长出来的

PHYB57 前面几章一直在算：数组、插值、求根、积分。算完之后留下的是一串浮点数，而没有人能盯着一万个浮点数看出规律。第 12 章就是把这一万个数变成一条曲线、一张曲面、一段动画的工具箱。

原课件只有九页幻灯片，列的基本是「有哪些函数可以调」。这份讲解版要补上幻灯片没写的那一半：这些函数背后的模型是什么、参数为什么要那样传，以及每个知识点在作业和实验报告里最容易翻车的地方。

这一章的四件事

01 二维绘图
用 `plot(x, y)` 把两个数组连成曲线，再修饰到「别人能看懂」的程度。

02 三维绘图
用 `projection='3d'` 打开第三个坐标轴，画参数曲线、曲面和散点。

03 地图
用 Cartopy 把经纬度数据放到一个真实的投影上。

04 动画
用 `matplotlib.animation` 把「随时间变化」本身画出来。

这四件事的难度不是线性递增的。真正的分水岭在第三件：二维和三维只是坐标轴多一根，而地图引入了**投影**——数据的坐标系和画布的坐标系第一次不再是同一个。把这件事想明白，Cartopy 的大部分报错就都能自己解释了。

画布的三层结构

matplotlib 所有的 API 都落在三层对象上。这三层分清楚，后面的函数名就不需要死记。

表 1 matplotlib 的三层对象模型

层	对象	它是什么	典型入口
最外层	Figure	整张画布，决定纸多大、多清晰	`plt.figure(figsize=(10, 6))`
中间层	Axes	画布上的一个坐标系，带 x 轴 y 轴（必要时还有 z 轴）	`plt.axes()` 、 `plt.subplots()`
最里层	Artist	坐标系里的每一个可见元素：线、点、文字、图例	`ax.plot()` 、 `ax.set_title()`

注意 **Axes 不是 axis**。`Axes` 是「一个坐标系」（可以理解成一张子图），`axis` 才是「一根坐标轴」。英文课件里这两个词只差一个字母，是最常见的阅读障碍之一。

要点 · 一句话抓住全章

几乎所有绘图代码都在做同一件事：先要到一个 `Axes`，再往这个 `Axes` 上挂 Artist。二维、三维、地图、动画的区别，只是这个 `Axes` 的类型不同，以及挂上去的 Artist 是静止的还是每帧更新的。

两套写法：状态机与面向对象

课件里混用了两种风格，初学时常常被绕晕。它们的区别只有一个：**当前坐标系是隐式的还是显式的**。

pyplot 状态机风格

```python
plt.figure(figsize=(10, 6))
plt.plot(x, y)
plt.title('Quadratic Function')
plt.xlabel('X axis')
plt.show()
```

`plt.` 开头的函数作用在「当前坐标系」上。写起来短，适合一次性画一张图。一旦出现多张子图，「当前」是谁就会变得含糊。

面向对象风格

```python
fig, ax =
plt.subplots(figsize=(10, 6))
ax.plot(x, y)
ax.set_title('Quadratic Function')
ax.set_xlabel('X axis')
plt.show()
```

坐标系握在 `ax` 这个变量里，想画到哪张图上就写哪个 `ax`。方法名多一个 `set_` 前缀。写动画和多子图时只能用这一套。

两种写法可以混用（课件就是这么干的），但建议：**练习和作业用状态机图省事，实验报告和动画一律用面向对象**，因为报告里几乎一定会出现多子图。

02

二维绘图：`plot(x, y)` 的真相

这是整章最被低估的一个函数。很多同学以为 `plt.plot` 会「画函数」，于是在 `plot(x, x**2)` 画得很漂亮时感觉良好，等到画 sin 时却得到一条折线，开始怀疑 matplotlib 坏了。

它连的是点，不是函数

`plot(x, y)` 接到的是两个等长的序列，它做的事情只有两步：把 (x_i, y_i) 一个个标到坐标系上，再用直线段依次连起来。**它完全不知道你心里那个函数是什么。**

所以曲线光不光滑，只取决于你给了多少个采样点。

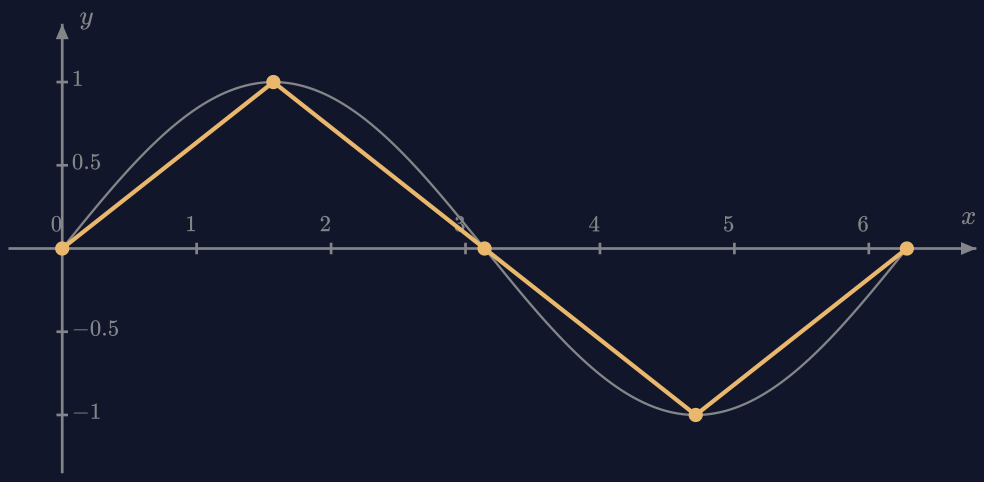


图 1　同一个 sin(x)：5 个采样点连出来是折线（亮色），400 个采样点才像曲线（灰色）。plot 从头到尾只是在连点。

课件里的 `x = np.linspace(-5, 5, 100)` 这一行，真正的作用不是「定义定义域」，而是**定下采样密度为 100**。把 100 改成 6，那条抛物线立刻变成折线。

提示 · 采样点该取多少

经验法则：函数在区间内有几个起伏，就至少给每个起伏 20 个点。画 $\sin x$ 在 $[0, 6\pi]$ 上有 3 个周期，`np.linspace(0, 6*np.pi, 200)` 足够；画高频振荡或尖峰函数则要几千点。课件动画例子里用 `n = 1000`，正是因为后面要把这 1000 个点当成 1000 帧。

格式字符串：`'ro'` 和 `'g--'` 怎么被解析

`plot(x, y, 'ro')` 的第三个参数叫 format string，是颜色、标记、线型三类字符的拼接，顺序可以任意，缺省的那一类走默认值。

表 2　格式字符串的三个组成部分（常用取值）

类别	常用字符	含义
颜色	`b` `g` `r` `c` `m` `y` `k` `w`	蓝 绿 红 青 品红 黄 黑 白（`k` 是 black，不是 `b`）
标记	`.` `o` `*` `s` `^` `+` `x`	点、实心圆、星、方块、三角、加号、叉
线型	`-` `--` `-.` `:`	实线、虚线、点划线、点线

于是课件里的几个例子就可以逐字翻译：

- `'ro'` = 红色 + 圆标记 + **没有线型** → 只画散点，不连线。
- `'g--'` = 绿色 + 虚线 + 没有标记 → 只有虚线，没有点。
- `'ko'` = 黑色实心圆，`'r*'` = 红色星号 —— 正是 Legend 那页里 quadratic 与 cubic 的画法。

最容易踩的坑：想要「又有点又有线」时，必须把线型也写进去，比如 `'ro-'`；只写 `'ro'` 得到的是一盘散沙。

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)
plt.plot(x, x**2, 'g--')       # 绿色虚线，无标记
plt.plot(x, x**2 - 10, 'ro')   # 红色圆点，无连线
plt.plot(x, x**2 - 20, 'b.-') # 蓝色，既有点也有实线
plt.show()
```

在 Notebook 里看图：inline、notebook 与 widget

课件提到两个魔法命令。它们改的是 matplotlib 的后端（backend），也就是「图最终交给谁来显示」。

- `%matplotlib inline`：把图渲染成静态位图嵌进输出单元。不能缩放、不能拖动，但稳定、导出 PDF 时一定在。
- `%matplotlib notebook`：启用交互后端，可以平移、缩放、读坐标。

> **注意 · 这条在 JUPYTERLAB 里不工作**
>
> `%matplotlib notebook` 是给**经典 Notebook** 用的。如果你用的是 JupyterLab 或 VS Code 的 notebook，正确写法是 `%matplotlib widget`，并且要先 `pip install ipympl`。写错了不会报语法错误，只会安静地退回静态图，很多人卡在这里以为自己装错了库。

交互后端还有一个副作用：同一个 figure 会一直「活着」等你操作，连着画好几张时容易串图。做作业时如果发现图叠在一起，先把后端换回 `inline` 排除干扰。

03

让图能被读懂：标题、标签、图例与样式

一张没有轴标签的图，在实验报告里等于没画。这一节的所有函数都只做一件事：把「作者脑子里的上下文」写到纸面上。

标题与轴标签

```python
plt.title('Quadratic Function')
plt.xlabel('X axis')
plt.ylabel('Y axis')
```

物理课里有个硬要求幻灯片没写：**轴标签必须带单位**。`plt.xlabel('Time (s)')`、`plt.ylabel('Velocity (m/s)')` 才是合格写法，只写 `x` 和 `y` 在实验报告里是要扣分的。

标签里可以直接写 LaTeX，用美元符号包起来即可：

```python
plt.xlabel(r'$t\ \mathrm{(s)}$')
plt.ylabel(r'$v\ \mathrm{(m\,s^{-1})}$')
plt.title(r'$v(t) = v_0 + at$')
```

前面那个 `r` 是 raw string，作用是让 `\m` 不被 Python 当成转义字符。忘了写 `r` 是最常见的「公式显示不出来」原因。

图形尺寸：`figsize` 与 `dpi` 是一个乘法

`plt.figure(figsize=(10, 6))` 的单位是**英寸**，不是像素。真正的像素数是：

$$\text{像素宽} = \text{figsize}[0] \times \text{dpi}, \qquad \text{像素高} = \text{figsize}[1] \times \text{dpi}$$

默认 `dpi = 100`，所以 `figsize=(10, 6)` 出来是 1000 × 600 像素。这个乘法还决定了另一件常被忽略的事：**字号是按英寸算的，不是按像素算的**。把 figsize 调大一倍，字相对就变小了一半，图会显得空旷；把 figsize 调小，字就挤成一团。

所以「图里的字太小」有两种解法，方向正好相反：缩小 `figsize`，或者直接调 `plt.rcParams['font.size']`。盲目放大 figsize 只会让字更小。

多条曲线与图例

```python
x = np.linspace(-5, 5, 100)
plt.plot(x, x**2, 'ko', label='Quadratic')
plt.plot(x, x**3, 'r*', label='Cubic')
plt.legend(loc=2)
plt.title('Plot of Various Polynomials from -5.0 to 5.0')
plt.show()
```

关键在于 `label=` 和 `legend()` 必须成对出现：`label` 是在每条曲线上贴名字，`legend()` 才是把这些名字收集起来画成图例框。只写 `label` 而不调 `legend()`，图例根本不会出现——这是本章最高频的作业失误。

表 3 `legend(loc=...)` 的位置编码

编号	位置	编号	位置
0	best（自动避让数据）	6	center left
1	upper right	7	center right
2	upper left	8	lower center
3	lower left	9	upper center
4	lower right	10	center
5	right		

课件用的 `loc=2` 就是左上角。实战中更推荐 `loc=0`（`'best'`），让 matplotlib 自己找数据最稀疏的角落；只有在图例被自动放到了碍事的位置时才手动指定。

全局样式

```python
plt.style.use('seaborn-poster')
```

样式表一次性改掉配色、线宽、字号、网格等一整套默认值，适合在报告开头统一全篇风格。

注意 · SEABORN 系样式已经改名

matplotlib 3.6 起，所有 `seaborn-*` 样式都加了版本前缀。现在的正确写法是 `plt.style.use('seaborn-v0_8-poster')`；直接照抄课件的 `'seaborn-poster'` 会抛 `OSError: 'seaborn-poster' is not a valid package style`。想看本机到底有哪些可用样式，运行 `print(plt.style.available)`。

04
三维绘图：从平面走进空间

二维的 `Axes` 换成三维的 `Axes3D`，画法几乎原样平移：`plot` 变 `plot3D`，`scatter` 多吃一个参数，新增一个二维专属没有的 `plot_surface`。

先要到一个三维坐标系

```python
import numpy as np
from mpl_toolkits import mplot3d
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(10, 10))
ax = plt.axes(projection='3d')
```

`projection='3d'` 是唯一的开关。课件里那行 `from mpl_toolkits import mplot3d` 在 matplotlib 3.2 之前是必需的（它负责把 `'3d'` 这个投影名注册进去），**现在已经可以省略**，matplotlib 会按需自动导入。保留着也无害，不保留也不会报错。

例 1：三维参数曲线

```python
t = np.arange(0, 10*np.pi, np.pi/50)
x = np.sin(t)
y = np.cos(t)
ax.plot3D(x, y, t)
ax.set_title('3D Parametric Plot')
plt.show()
```

这段代码画出来是一根**螺旋线**，原因值得想清楚：$x = \sin t$ 与 $y = \cos t$ 合起来满足 $x^2 + y^2 = 1$，也就是说投影到 xy 平面上永远是同一个单位圆；而第三个坐标直接取 t ，随时间匀速上升。圆周运动 + 匀速上升 = 螺旋。

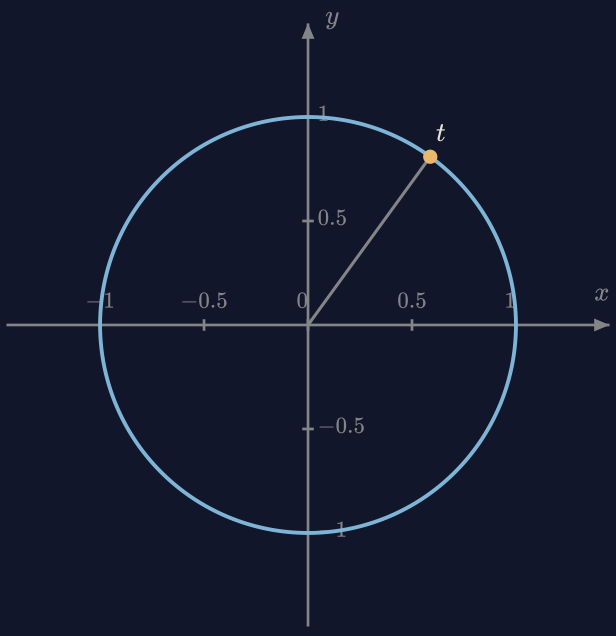


图 2 把螺旋线压扁到 xy 平面：x=sin t、y=cos t 画出的始终是单位圆，高度全部交给第三个坐标 t。

`np.arange(0, 10*np.pi, np.pi/50)` 给出 500 个采样点，覆盖 5 整圈。`plot3D` 其实就是 `Axes3D.plot` 的别名，两个名字完全等价。

`meshgrid`：把两个一维数组摊成网格

这是整章最容易含糊的概念，也是曲面图能不能跑起来的关键。`plot_surface` 要求 `X`、`Y`、`Z` 三个都是**二维**数组，而我们手上通常只有两条一维的轴。`np.meshgrid` 负责这个升维。

```python
x = np.array([0, 1, 2])
y = np.array([10, 20])
X, Y = np.meshgrid(x, y)
```

表 4 `meshgrid` 做了什么（`x` 有 3 个值、`y` 有 2 个值，结果都是 2×3）

数组	内容	规律
`X`	`[[0, 1, 2], [0, 1, 2]]`	把 `x` 按行复制，复制 `len(y)` 次
`Y`	`[[10, 10, 10], [20, 20, 20]]`	把 `y` 按列复制，复制 `len(x)` 次

把两张表叠在一起看，`(X[i][j], Y[i][j])` 恰好遍历了网格上的每一个交点。这样 `Z = np.sin(X) * np.cos(Y)` 这种**逐元素**运算就能一次算出整片曲面的高度，不需要写双重循环。

注 · 形状的那个反直觉

默认 `indexing='xy'` 下，`X.shape` 和 `Y.shape` 都是 `(len(y), len(x))` ——行数由 `y` 决定，列数由 `x` 决定，和直觉正好相反。只要始终用 `meshgrid` 的输出算 `Z`，就不会出问题；一旦自己手搓 `Z` 的形状，`ValueError: shape mismatch` 就会找上门。

例 2：曲面图

```python
x = np.arange(-5, 5.1, 0.2)
y = np.arange(-5, 5.1, 0.2)
X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)

fig = plt.figure(figsize=(10, 8))
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, cmap=plt.cm.cividis)
ax.set_xlabel('x'); ax.set_ylabel('y'); ax.set_zlabel('z')
plt.show()
```

课件那张图像个「鸡蛋托盘」，道理藏在乘法里：固定住 y ，$Z = \sin X \cdot \cos y$ 就是一条振幅为 $\cos y$ 的正弦曲线。换一个 y ，振幅跟着变，甚至在 $\cos y < 0$ 时整条曲线上下翻转。把这些切片并排放好，就是那片起伏。

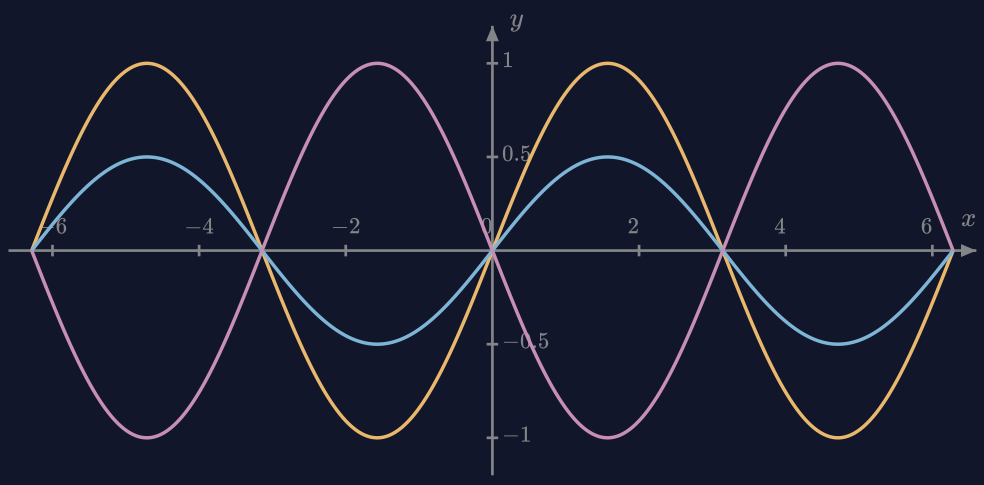


图 3 曲面 `Z = sin(X)cos(Y)` 的三条切片：`cos y = 1`、`0.5`、`-1`。振幅随 `y` 变化，负值时整条翻转——这就是鸡蛋托盘的来历。

注意 · NP.ARANGE 的端点是不可靠的

`np.arange(-5, 5.1, 0.2)` 写成 `5.1` 而不是 `5` 是有意的：`arange` 的终点**不包含**在内，而且步长是浮点数时，最后一个点到底取不取到受舍入误差影响。网格点数要精确可控时，一律用 `np.linspace(-5, 5, 51)` ——它的端点一定包含，点数一定是你写的那个数。

例 3：三维散点

```python
x = np.random.random(50)
y = np.random.random(50)
z = np.random.random(50)

ax = plt.axes(projection='3d')
ax.scatter(x, y, z, c='r', s=50)
plt.show()
```

`c` 是颜色，`s` 是标记面积（单位是 point²，不是半径）。`c` 还可以传一个和数据等长的数组，配合 `cmap` 把第四个变量编码成颜色——报告里想同时展示四个维度时就靠这招。

配色表怎么选

课件用的 `plt.cm.cividis` 不是随手挑的。

提示 · 三条选色谱的规矩

连续数据用 `viridis`、`cividis`、`plasma` 这类感知均匀的色谱：颜色变化的快慢和数值变化成正比，眼睛不会被假的「条纹」骗到。其中 `cividis` 还对红绿色盲友好。**有正负的数据**（比如 $\sin X \cos Y$）用 `coolwarm`、`RdBu` 这类双向色谱，让零值落在中性色上。**务必避开 `jet`**：它在青绿区域制造虚假的亮度跳变，在科研制图里已经被明确不推荐。

05

地图：把数据放回地球上

地图是一张特殊的二维图：横轴是经度、纵轴是纬度，但这两条轴之间的换算规则（投影）要由一个专门的库来管。

三个包，一段历史

2016 前 — **Basemap**

matplotlib 官方的地图扩展，功能齐全但依赖沉重，2016 年后停止维护，现在只会在老代码里见到。

2016 后 — **Cartopy**

Basemap 的官方继任者，由英国气象局主导，投影体系完整，和 matplotlib 深度集成。本章用的就是它。

同期 — **Folium**

另一条路线：把数据交给浏览器里的 Leaflet.js，产出可拖动可缩放的网页地图。适合展示，不适合排进 PDF。

经纬度：先把约定钉死

表 5 经纬度的取值范围与符号约定

量	英文	范围	符号	在图上的位置
经度	longitude	$-180^\circ \sim 180^\circ$	东经为正，西经为负	横轴（x）
纬度	latitude	$-90^\circ \sim 90^\circ$	北纬为正，南纬为负	纵轴（y）

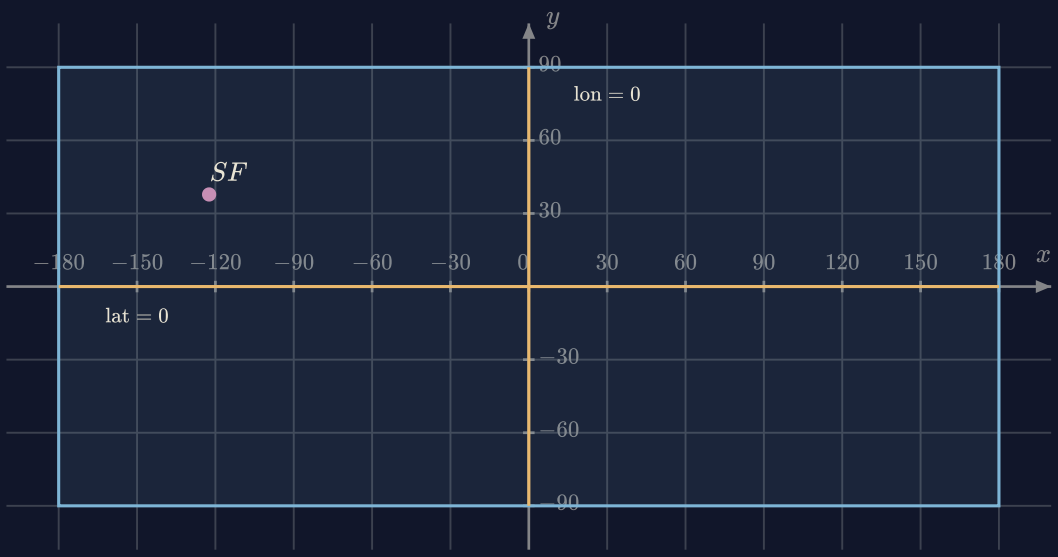


图 4 Plate Carree 投影下的整个地球：经度直接当 x、纬度直接当 y，铺成一个 360x180 的矩形。旧金山在 (-122.4, 37.8)——经度是负的，因为它在西半球。

顺序永远是 (经度, 纬度)，也就是先 x 后 y。日常说「北纬 37 度、西经 122 度」时习惯先说纬度，代码里必须反过来写，这是坐标写反的主要来源。

最小可用地图

```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))
ax = plt.axes(projection=ccrs.PlateCarree())
ax.coastlines()
ax.gridlines(draw_labels=True)
plt.show()
```

`ccrs` 是 **c**artopy **c**oordinate **r**eference **s**ystem 的缩写。Plate Carrée（等距圆柱投影）是最简单的一种：经度直接当横坐标、纬度直接当纵坐标，所以整个地球铺成一个 2:1 的矩形。它面积严重失真（格陵兰看起来和非洲差不多大），但胜在直观，是入门和做示意图的默认选择。

课件提到的「用降采样的 Natural Earth 底图改善背景」，对应的是一行 `ax.stock_img()`——它会铺上一张低分辨率的全球地貌影像，让空白的海陆变成有颜色的地球。

`transform=` 这个坑

注意 · 画数据时一定要写 TRANSFORM

在地图上画自己的点和线，必须显式声明「我这组坐标是什么坐标系的」：

```python
ax.plot(-122.4, 37.8, 'ro', transform=ccrs.PlateCarree())
```

`projection=` 说的是**画布**用什么投影，`transform=` 说的是**数据**用什么坐标系，两者是独立的。省略 `transform` 时，cartopy 会把你的经纬度当成投影后的平面坐标，点就会飞到地图外面去。只要养成「经纬度数据恒配 `transform=ccrs.PlateCarree()`」的习惯，这个坑就永远不会踩。

06

Cartopy 进阶：区域、要素与分辨率

全球图只是起点。真正有用的地图总是「某一块区域 + 几类地理要素 + 合适的精度」。

`set_extent`：框出你要的那块

```python
ax.set_extent([-122.8, -122, 37.3, 38.3])
```

参数顺序是 `[经度最小，经度最大，纬度最小，纬度最大]`——**两个经度在前，两个纬度在后**，不是按 (lon, lat) 成对排的。课件这组数字框的正是旧金山湾区。

函数还有第二个参数 `crs`，省略时按**坐标轴自身的投影**解释这四个数。当坐标轴就是 `PlateCarree` 时无所谓；一旦换成墨卡托之类的投影，就必须写成 `ax.set_extent([...], crs=ccrs.PlateCarree())`，否则那四个数会被当成米而不是度。

`add_feature`：往地图上加地理要素

```python
import cartopy.feature as cfeature

ax.add_feature(cfeature.LAND)
ax.add_feature(cfeature.OCEAN)
ax.add_feature(cfeature.BORDERS)
ax.add_feature(cfeature.LAKES, alpha=0.5)
ax.add_feature(cfeature.RIVERS)
```

表 6 Cartopy 常用内置要素

要素	画出来的东西	常见用法
`LAND` / `OCEAN`	陆地与海洋的填充色	先铺这两层当底
`COASTLINE`	海岸线	等价于 `ax.coastlines()`
`BORDERS`	国界线	默认只有国界，不含省界
`LAKES`	湖泊	配 `alpha=0.5` 做半透明
`RIVERS`	河流	小比例尺下密得发黑，慎用

`alpha=0.5` 是透明度（0 全透明、1 不透明），用来让下面那层还能透出来。要素的叠放顺序就是调用顺序，先画的在下面——所以 `LAND` 必须写在 `BORDERS` 前面，否则国界会被陆地盖住。

分辨率：三档精度

Natural Earth 的数据分三档，名字来自地图比例尺：

110m
默认档
1:110,000,000，只够画全球轮廓

50m
中等档
1:50,000,000，适合大洲和国家

10m
最精细
1:10,000,000，城市级区域图必须用这档

数字越小越精细（分母越小、比例尺越大），这一点和「10m 听起来像 10 米」的直觉完全无关。课件里那张湾区图之所以能看出海湾的形状，就是因为用了 `10m`：

```python
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
ax = plt.axes(projection=ccrs.PlateCarree())
ax.set_extent([-122.8, -122, 37.3, 38.3])

ax.coastlines(resolution='10m')
ax.add_feature(cfeature.LAND.with_scale('10m'))
ax.add_feature(cfeature.OCEAN.with_scale('10m'))
ax.add_feature(cfeature.LAKES.with_scale('10m'), alpha=0.5)
plt.show()
```

用默认的 `110m` 画这块区域，整个湾区会退化成几根折线。注意 `coastlines()` 用 `resolution=` 参数，而 `cfeature.*` 用 `.with_scale()` 方法，两套写法别混。

注 · 第一次运行会很慢

Cartopy 的地理数据不随包一起安装，而是在第一次用到某个分辨率时现场从 Natural Earth 下载并缓存到本地。所以第一次跑 `10m` 的代码要等上几十秒，之后就快了。在没有网络的考场机器上，这意味着代码会直接报下载失败——提前跑一遍把缓存建好。

07

动画：让时间成为第三个维度

静态图只能展示一个时刻。要展示一个随时间演化的系统——振动、轨道、波的传播——动画是最诚实的表达方式。

动画的本质

一段动画就是一叠快速翻过的静止画面。在 matplotlib 里，这件事被拆成两半：

- **背景**：坐标轴范围、标签、不动的那条正弦曲线——只画一次。
- **前景**：那个要移动的红点——每一帧改一次它的数据。

关键在于**不要每帧都重画整张图**，而是只更新会变的那几个 Artist。`set_data()` 就是干这个的：它换掉一条线的 x/y 数据，而不新建对象。

1000	15 fps	66.7 秒
帧数	**播放速率**	**成片时长**
课件例子里 `n = 1000`	`writer` 的 `fps` 参数	`1000 ÷ 15 = 66.7`

路线 A：`FuncAnimation`

这是在屏幕上实时播放的写法，也是写交互式 notebook 时的首选。

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation

n = 1000
x = np.linspace(0, 6*np.pi, n)
y = np.sin(x)

fig, ax = plt.subplots()
ax.set_xlim(0, 6*np.pi)
ax.set_ylim(-1.5, 1.5)
ax.set_xlabel('x'); ax.set_ylabel('sin(x)')

sine_line, = ax.plot(x, y, 'b')                    # 背景：蓝色正弦曲线
red_circle, = ax.plot([], [], 'ro', markersize=10)    # 前景：空的红点

def init():
    red_circle.set_data([], [])
    return red_circle,

def update(frame):
    red_circle.set_data([x[frame]], [y[frame]])
    return red_circle,

ani = animation.FuncAnimation(
    fig, update, frames=n, init_func=init, interval=15, blit=True)
plt.show()
```

注意 `sine_line, = ax.plot(...)` 结尾那个**逗号**。`plot()` 返回的是一个列表（可能画了多条线），这个逗号是元组解包，把列表里唯一的那条线取出来。漏掉逗号，后面调 `set_data()` 时就会报 `AttributeError: 'list' object has no attribute 'set_data'`。

路线 B：`writer.saving()` 直接写 mp4

课件最后一页用的是另一条路：不播放，直接一帧一帧写进视频文件。

```python
import matplotlib.animation as manimation

FFMpegWriter = manimation.writers['ffmpeg']
metadata = dict(title='Sine Wave Animation', artist='Matplotlib',
                comment='A red circle moving along a sine wave')
writer = FFMpegWriter(fps=15, metadata=metadata)

with writer.saving(fig, 'sine_wave_animation.mp4', 100):
    for i in range(n):
        red_circle.set_data([x[i]], [y[i]])   # 更新红点位置
        writer.grab_frame()                   # 把当前画面抓成一帧

print('Animation saved as sine_wave_animation.mp4')
```

`writer.saving(fig, 文件名, dpi)` 的第三个参数 `100` 是 **dpi**，不是帧数也不是质量。配合 `figsize` 决定视频分辨率：默认 `figsize=(6.4, 4.8)` 加 dpi=100，出来是 640×480。

表 7 两条路线怎么选

	`FuncAnimation`	`writer.saving()`
产出	屏幕上实时播放，也可再 `.save()`	直接得到 mp4 文件
帧循环	由 matplotlib 调度，你只写 `update`	自己写 `for` 循环
适合	notebook 里边看边调	批量生成、交作业附视频
必须的外部依赖	无（只播放时）	ffmpeg

`blit`、`interval` 与 `fps`

- `interval=15`：**播放**时两帧之间的毫秒数，15 ms 对应约 67 帧/秒。它只影响屏幕上的速度。
- `fps=15`：**保存**时写进文件的帧率。它只影响视频文件的速度。两者互不干涉，可以不一致。
- `blit=True`：只重绘发生变化的那一小块区域，能让动画流畅好几倍。代价是 `update` 和 `init` **必须返回一个可迭代的 Artist 集合**——代码里 `return red_circle,` 末尾那个逗号就是为此存在的，它把单个对象变成只有一个元素的元组。返回 `red_circle`（没有逗号）会直接报错。

注意 · 两个必踩的坑

第一，动画对象必须留一个引用。 写成 `animation.FuncAnimation(...)` 而不赋值给变量，Python 的垃圾回收会立刻把它收掉，你会看到一张静止的图，而且**没有任何报错**。一定写 `ani = animation.FuncAnimation(...)`。

第二，ffmpeg 要单独装。 `manimation.writers['ffmpeg']` 找不到可执行文件时会抛 `RuntimeError: Requested MovieWriter (ffmpeg) not available`。macOS 用 `brew install ffmpeg`，Windows 可以 `conda install -c conda-forge ffmpeg`。实在装不上就退而求其次存 GIF：把 writer 换成 `manimation.PillowWriter(fps=15)`，Pillow 是 matplotlib 的自带依赖，一定在。

08

速查、常见坑与自测

函数速查表

表 8 本章出现的全部函数

函数	作用	要点
`plt.plot(x, y, fmt)`	二维折线/散点	fmt = 颜色+标记+线型
`plt.figure(figsize=)`	新建画布	单位是英寸
`plt.title/xlabel/ylabel`	标题与轴标签	物理图必须带单位
`plt.legend(loc=)`	图例	必须先给 `label=`
`plt.style.use()`	全局样式	seaborn 系已改名
`plt.axes(projection='3d')`	三维坐标系	三维的唯一开关
`ax.plot3D(x, y, z)`	三维曲线	就是 `ax.plot`
`ax.scatter(x, y, z, c=, s=)`	三维散点	`s` 是面积
`np.meshgrid(x, y)`	一维轴 → 二维网格	形状是 `(len(y), len(x))`
`ax.plot_surface(X, Y, Z, cmap=)`	曲面	三个参数都要二维
`plt.axes(projection=ccrs.*)`	地图坐标系	`ccrs` = cartopy 投影
`ax.coastlines(resolution=)`	海岸线	`'110m'/'50m'/'10m'`
`ax.gridlines(draw_labels=)`	经纬网格	带标签才好读
`ax.set_extent([...])`	限定区域	顺序 lon,lon,lat,lat
`ax.add_feature(cfeature.*)`	地理要素	先调的在下层
`FuncAnimation(...)`	动画	返回值必须留引用
`writer.saving(fig, 名, dpi)`	存视频	第三参数是 dpi

十个最常见的坑

只有 label 没有 legend	曲线名字贴好了却不调 `plt.legend()`，图例不会自己出现。
'ro' 却想要连线	格式串里没写线型就没有线，要写 `'ro-'`。
'b' 当成 black	`b` 是 blue，black 是 `k`。
linspace 点数太少	曲线变折线，不是函数算错了。
忘了 raw string	轴标签里写 LaTeX 不加 `r''`，反斜杠被 Python 吃掉。
seaborn-poster 报错	新版要写 `'seaborn-v0_8-poster'`。
Z 的形状不对	`plot_surface` 的三个参数都必须是 meshgrid 出来的二维数组。
地图上点飞出去	画经纬度数据漏了 `transform=ccrs.PlateCarree()`。
set_extent 顺序写反	是 lon_min, lon_max, lat_min, lat_max。
动画变成静止图	`FuncAnimation` 的返回值没有赋给变量，被垃圾回收了。

自测题

试着先答再看答案。

例 · 五道题

1. `plt.plot(np.linspace(0, 2*np.pi, 5), np.sin(np.linspace(0, 2*np.pi, 5)))` 画出来是什么形状?

2. `'k*--'` 这个格式串表示什么?

3. `x` 有 60 个元素、`y` 有 40 个元素，`X, Y = np.meshgrid(x, y)` 之后 `X.shape` 是多少?

4. 在 `ccrs.PlateCarree()` 的坐标轴上画多伦多（约 79.4° W, 43.7° N），这一行该怎么写?

5. `FuncAnimation` 里 `interval=20` 和 writer 的 `fps=25`，哪个决定导出 mp4 的播放速度?

小结 · 答案

1. 一条由 (0,0)、(π/2,1)、(π,0)、(3π/2,−1)、(2π,0) 五点连成的**折线**，像个三角波——采样太疏，不是正弦。

2. 黑色（`k`）、星形标记（`*`）、虚线（`--`），三者同时出现。

3. `(40, 60)` ——行数由 `y` 的长度决定，列数由 `x` 的长度决定。

4. `ax.plot(-79.4, 43.7, 'ro', transform=ccrs.PlateCarree())`。经度在前且为负（西经），并且必须带 `transform`。

5. `fps=25`。`interval` 只管屏幕上的实时播放速度，导出文件的帧率由 writer 的 `fps` 决定。

要点 · 带走这三句

第一，`plot` 只会连点，曲线的光滑程度由你给的采样点数决定。第二，`plot_surface` 的三个参数必须都是 `meshgrid` 产出的二维数组，`Z` 靠逐元素运算一次算完。第三，地图上 `projection=` 管画布、`transform=` 管数据，经纬度数据永远配 `transform=ccrs.PlateCarree()`。